

INTRODUCTION

Database 2

Advanced Database Systems

Data Models

- **Relational Model** uses a collection of tables to represent both data and the relationships among those data.
- Table is consisted of **multiple columns**, and each column has a unique name. **Tables are also known as relations.**
- **Entity-Relationship Model (E-R)** data model uses a collection of basic objects, called *entities*, and *relationships* among these objects.
- **An entity is a “thing” or “object”** in the real world that is distinguishable from other objects. The entity-relationship model is widely used in database design
- **Semi-structured Data Model**, permits the specification
- of data where individual data items of the same type may have different sets of attributes.

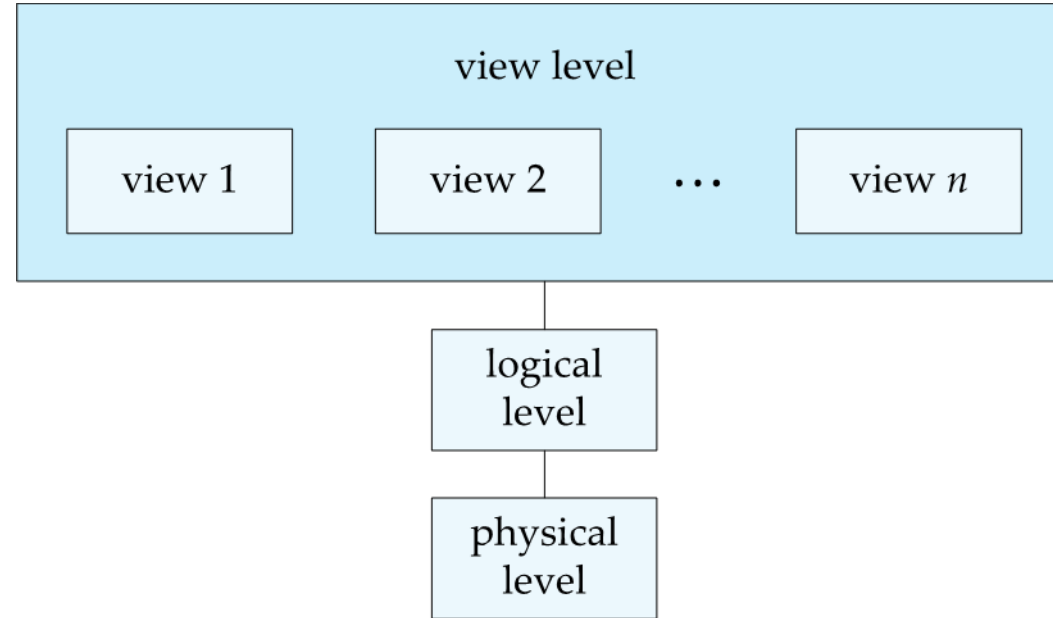
Data Models (2)

- **Semi-structured Data Model**, permits the specification of data where individual data items of the same type may have different sets of attributes.
- **This is in contrast to the data models mentioned earlier**, where every data item of a particular type must have the same set of attributes.
- ***JSON* and *Extensible Markup Language (XML)*** are widely used semi-structured data representations.

Data models (3)

- **Object-Based Data Model** (especially in Java, C++, or C#) is the dominant software-development methodology.
- Today the concept of objects is **well integrated into relational databases**.
- Standards exist to store objects in relational tables.
- Its seen as extending the **relational model with notions of encapsulation, methods, and object identity**.

Views



Levels of Abstraction

- **Physical level:** describes how a record (e.g., instructor) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data.
- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

Instances and Schemas

- The collection of information stored in the database at a particular moment is called an **instance of the database**.
- The overall design of the database is called **the database schema**.
- **The physical schema** describes the database design at the physical level.
- **The logical schema** describes the database design at the logical level.
- A database may also have several schemas at the view level called **subschemas**, that describe different views of the database.

SQL – Structured Query Language

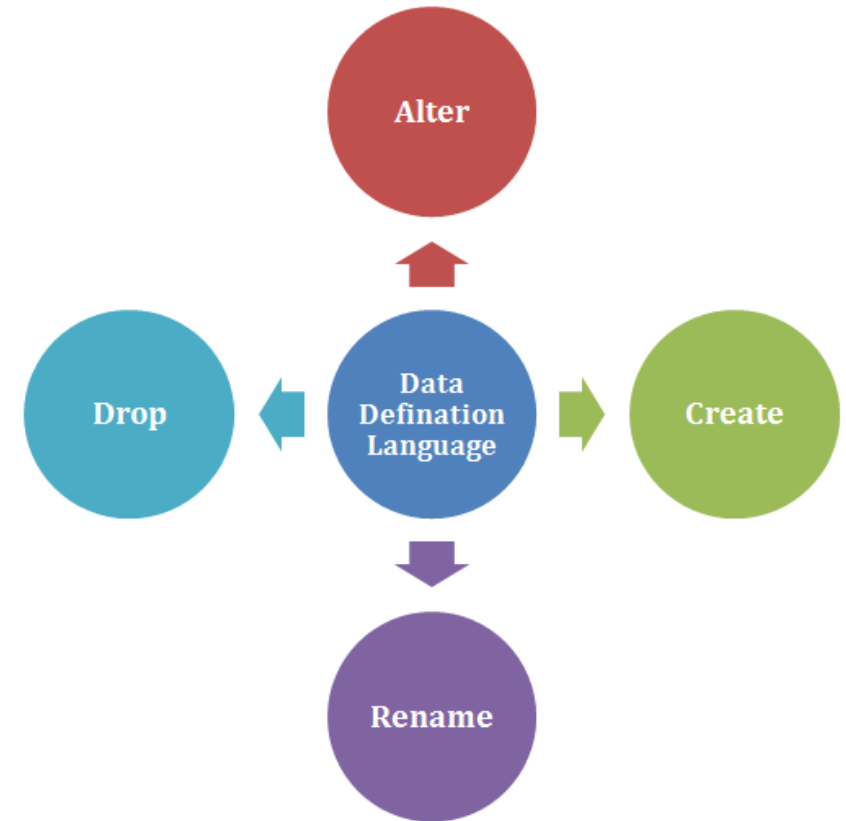
- SQL query language is nonprocedural.
- A query takes as input several tables (possibly only one) and always returns a single table.
- SQL is **NOT** a Turing machine equivalent language
- To be able to compute complex functions SQL is usually embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - API - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database
- **Open Database Connectivity (ODBC)** – created by Microsoft to access the database via SQL
- **Java Database Connectivity (JDBC)** – created by Sun Microsystems to enable Java applications to use SQL for database

Database Languages (1)

- **Data Definition Language (DDL)** – specify the database schema
 - We specify the storage structure and access methods used by the database system
 - by a set of statements in a special type of **DDL called a data storage and definition language**.
 - DDL compiler generates a set of table templates stored in a *data dictionary*
- **Data dictionary contains metadata (i.e., data about data)**
 - Database schema
 - Integrity constraints
 - Primary key (ID uniquely identifier)
 - Authorization
 - Who can access what

Data Definition Language (DDL)

- Subset of SQL used to define and manage database structures.
- Focuses on creating, altering, and deleting database objects (tables, schemas, indexes, etc.).
- Common DDL commands:
 - CREATE – build new database objects.
 - RENAME – modify existing objects.
 - DROP – delete objects.
 - ALTER - remove all records from a table quickly
- Executes with auto-commit → changes are permanent.



Database Languages (2)

Data Manipulation Language (DML) - accessing and updating the data organized by the appropriate data model

- There are basically two types of data-manipulation language
 - **Procedural DML** -- require a user to specify what data are needed and how to get those data.
 - **Declarative DML** -- require a user to specify what data are needed without specifying how to get those data.
- The portion of a DML that involves information retrieval is called a **query** language.

Data Manipulation Language (DML)

- Subset of SQL used to manage and manipulate data within database objects.
- Focuses on inserting, updating, deleting, and retrieving data.
- Does not define database structure (handled by DDL).
- Common DML commands:
 - SELECT – retrieve data from tables.
 - INSERT – add new records.
 - UPDATE – modify existing records.
 - DELETE – remove specific records.
- Transactions possible → changes can be rolled back or committed.

DDL & DML Commands

DDL COMMANDS

CREATE

RENAME

TRUNCATE

ALTER

DROP

DML COMMANDS

SELECT

INSERT

DELETE

UPDATE

DDL

DDL stands for Data Definition Language.

It helps us to define the structure of the database.

It affects the whole table.

It helps in defining the fields or columns of the tables.

They are auto-committed (changes made are permanent).

Changes are permanent.

They don't have further classification.

Commands used: CREATE, ALTER, TRUNCATE, RENAME, DROP.

DML

DML stands for Data Manipulation Language.

It helps us to manage the data.

It only affects the specified row(s) of the table.

It helps us in defining rows or records of the tables.

They are not auto-committed (changes are not permanent).

We can roll back our changes in the DML language

They are classified into procedural and non-procedural.

Commands used: SELECT, UPDATE, DELETE, INSERT.

Relational Models Recap (1)

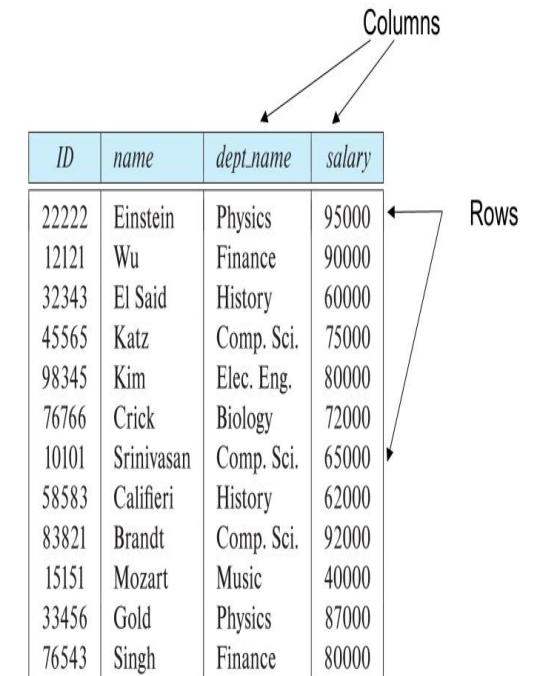
- Definition: **A way to represent data using tables (relations).**
- Core elements:
 - **Relation (Table):** set of rows (tuples) and columns (attributes).
 - **Tuple (Row):** represents a single record.
 - **Attribute (Column):** represents a property/field of the data.
- **Primary Key:** uniquely identifies each tuple.
- **Foreign Key:** enforces relationships between tables.

Relational Models Recap (2)

- Properties:
 - Each table has a unique name.
 - Column values come from defined domains (data type)
 - Order of tuples/attributes is irrelevant.
 - Integrity constraints maintain data consistency.
- Benefits:
 - Simple, table-based structure.
 - Strong foundation for SQL.
 - Supports normalization → reduces redundancy.
 - Ensures consistency through constraints.

Relational Model

- All the data is stored in various tables.
- Example of tabular data in the relational model



The diagram shows a table with four columns: ID, name, dept_name, and salary. The first row is highlighted in light blue. Arrows point from the word 'Columns' to the column headers, and from the word 'Rows' to the first row of data.

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califeri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

ER Modeling

- ER Diagrams: visual representation of entities and their relationships.
- Entities & Attributes: objects (tables) and their properties (columns).
- Relationships: define how entities are connected (1:1, 1:N, M:N).
- Keys
 - Primary Key → uniquely identifies an entity.
 - Foreign Key → links related entities.



Normalization

Aims to **reduce redundancy and improve data consistency**.

Functional Dependencies: represent the **relationship between attributes**.

Normal forms overview:

- **1NF**: eliminate repeating groups, ensure atomic values.
- **2NF**: remove partial dependencies (*attribute depends on part of a composite key*)
- **3NF**: remove transitive dependencies (attribute depends on another attribute, not the primary key)

Student_Course_Enrollment table

analyze keys, constraints, and normalization

StudentID	StudentName	CourseID	CourseName	InstructorName	InstructorEmail	Grade
101	Ana Petrova	CSE101	Database Systems	Dr. Mark Smith	mark.smith@univ.edu	A
102	Ivan Jovan	CSE101	Database Systems	Dr. Mark Smith	mark.smith@univ.edu	B
101	Ana Petrova	CSE202	AI Fundamentals	Dr. Sara Lee	sara.lee@univ.edu	A-
103	Mila Kovač	CSE202	AI Fundamentals	Dr. Sara Lee	sara.lee@univ.edu	B+
104	Petar Iliev	CSE303	Data Structures	Dr. John Brown	john.brown@univ.edu	A

Advanced SQL: Joins

Key Types of Joins:

- INNER JOIN – returns only matching rows from both tables.
- LEFT JOIN – returns all rows from the left table + matches from the right.
- RIGHT JOIN – returns all rows from the right table + matches from the left.
- FULL OUTER JOIN – returns all rows from both tables (matches + non-matches).

Example

- SELECT s.StudentName, c.CourseName
- FROM Students s
- INNER JOIN Enrollments e ON s.StudentID = e.StudentID
- INNER JOIN Courses c ON e.CourseID = c.CourseID;

Advanced SQL: Subqueries

- **Subquery Types:**
- **Scalar Subquery** – returns a single value.
- **Row Subquery** – returns one row.
- **Table Subquery** – returns multiple rows.
- **Correlated Subquery** – depends on outer query values.

Example

- SELECT StudentName
- FROM Students
- WHERE StudentID IN (
 - SELECT StudentID FROM Enrollments WHERE Grade = 'A'
-);

Advanced SQL: Aggregation

- **Aggregate Functions:**
- COUNT(), SUM(), AVG(), MIN(), MAX()
- Combine with **GROUP BY** and **HAVING**

Example:

```
SELECT CourseID, AVG(Grade) AS AvgGrade  
FROM Enrollments  
GROUP BY CourseID  
HAVING AVG(Grade) > 3.0;
```

Advanced SQL: Window Functions

Common Window Functions:

- **ROW_NUMBER()** – assigns sequential number.
- **RANK()** / **DENSE_RANK()** – ranking with ties.
- **LEAD()** / **LAG()** – access next/previous row.

Example:

```
SELECT StudentID, Grade,  
       RANK() OVER (PARTITION BY CourseID ORDER BY Grade DESC) AS  
       RankInCourse  
FROM Enrollments;
```

Advanced SQL: Transactions & ACID

- **Transactions:** group of SQL statements executed as a unit.
- **ACID Properties:** Atomicity, Consistency, Isolation, Durability.
- **Transaction Control Commands:** COMMIT, ROLLBACK, SAVEPOINT.

Example:

```
BEGIN TRANSACTION;
```

```
UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;
```

```
UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;
```

```
COMMIT;
```

Advanced SQL: Isolation Levels

- **READ UNCOMMITTED** – allows dirty reads.
- **READ COMMITTED** – prevents dirty reads (default in many DBs).
- **REPEATABLE READ** – consistent rows during a transaction.
- **SERIALIZABLE** – strictest, prevents phantom reads.

Advanced SQL: Practice & Discussion

- **Joins:**
 - Write a query to list all students who are *not enrolled* in any course.
- **Subqueries:**
 - Find students whose grades are *above the average grade* in their course.
- **Aggregation:**
 - Which course has the *highest average grade*?
- **Window Functions:**
 - Assign a rank to students within each course based on their grade.
- **Transactions:**
 - Give a *real-life example* where **ROLLBACK** would be critical.
- **Isolation Levels:**
 - *Why might a bank use SERIALIZABLE isolation for transfers but READ COMMITTED for balance inquiries?*

Write a query to list all students who are *not enrolled* in any course?

- SELECT s.StudentName
- FROM Students s
- LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
- WHERE e.StudentID IS NULL;

StudentID	StudentName
101	Ana Petrova
102	Ivan Jovan
103	Mila Kovač
104	Petar Iliev

StudentID	CourseID	Grade
101	CSE101	A
102	CSE101	B
101	CSE202	A-
103	CSE202	B+
104	CSE303	A

Find students whose grades are *above the average grade* in their course.

- SELECT s.StudentName, e.Grade, e.CourseID
- FROM Enrollments e
- JOIN Students s ON e.StudentID = s.StudentID
- WHERE e.Grade > (
 - SELECT AVG(Grade)
 - FROM Enrollments e2
 - WHERE e2.CourseID = e.CourseID
 -);

StudentID	CourseID	Grade
101	CSE101	A
102	CSE101	B
101	CSE202	A-
103	CSE202	B+
104	CSE303	A

StudentID	StudentName
101	Ana Petrova
102	Ivan Jovan
103	Mila Kovač
104	Petar Iliev

Which course has the *highest average grade*?

- SELECT CourseID, AVG(Grade) AS AvgGrade
- FROM Enrollments
- GROUP BY CourseID
- ORDER BY AvgGrade DESC
- LIMIT 1;

StudentID	CourseID	Grade
101	CSE101	A
102	CSE101	B
101	CSE202	A-
103	CSE202	B+
104	CSE303	A

CourseID	CourseName	InstructorID
CSE101	Database Systems	INS1
CSE202	AI Fundamentals	INS2
CSE303	Data Structures	INS3

Assign a rank to students within each course based on their grade.

- SELECT StudentID, CourseID, Grade,
- RANK() OVER (PARTITION BY CourseID ORDER BY Grade DESC) AS RankInCourse
- FROM Enrollments;

StudentID	CourseID	Grade
101	CSE101	A
102	CSE101	B
101	CSE202	A-
103	CSE202	B+
104	CSE303	A

StudentID	StudentName
101	Ana Petrova
102	Ivan Jovan
103	Mila Kovač
104	Petar Iliev

Enrollment (joined with Students for names if needed)

Give a *real-life example* where ROLLBACK would be critical.

- *Scenario 1:*
- Online store checkout: If payment succeeds but inventory update fails, ROLLBACK ensures no money is charged without stock being reserved.
- *Scenario 2:*
- Airline Booking System: A passenger books a flight seat:
 1. System reserves the seat and
 2. System processes the payment.
 3. System updates loyalty points.If step 3 fails (e.g., loyalty system error), the transaction should ROLLBACK to avoid charging the customer without confirming the seat.

Why might a bank use **SERIALIZABLE** isolation for transfers but **READ COMMITTED** for balance inquiries?

- **Transfers: require **SERIALIZABLE**** to prevent double spending (phantom or dirty reads would be dangerous).
- **Balance inquiries: can use **READ COMMITTED****, since small anomalies don't affect overall safety and it's faster.